

Sim-to-Real Navigation with A2C and DAgger

Reinforcement Learning Laboratory 4 – 2026-I

Ricardo Amiel Acuña Villogas
Data Science
UTEC, Lima, Peru

Salvador Justo Ruben Cordova Flores
Computer Science
UTEC, Lima, Peru

Rodrigo Félix Meza Gavilán
Computer Science
UTEC, Lima, Peru

I. MDP FORMULATION

The navigation task is modeled as $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

State space $s_t \in \mathbb{R}^{14}$:

$$s_t = \left[\underbrace{d_n, \Delta\theta_n, \Delta x_n, \Delta y_n}_{\text{goal info (4)}}, \underbrace{l_1, \dots, l_{10}}_{\text{LiDAR} \in [0,1] \text{ (10)}} \right]$$

where $d_n = d/5$ m, $\Delta\theta_n = \Delta\theta/\pi$, and the 10 LiDAR rays (180° FOV, range 5 m) are computed geometrically from obstacle positions and normalized to $[0, 1]$.

Action space $a_t = (v_t, \omega_t)$ is continuous:

$$v_t \in [0, 0.22] \text{ m/s}, \quad \omega_t \in [-1, 1] \text{ rad/s}$$

Notice that the real speed of the vehicle at our environment in Gazebo is around 0.3-0.8 m/s, but a real turtlebot (3/4) has a speed of 0.22 m/s.

The Gaussian actor outputs $\mu \in [-1, 1]^2$ via Tanh, and real commands are $v = \frac{\mu_0+1}{2} \cdot v_{\max}$, $\omega = \mu_1$.

Reward function:

$$r_t = \underbrace{2(d_{t-1} - d_t)}_{\text{progress}} + \underbrace{0.05 \left(1 - \frac{|\Delta\theta|}{\pi}\right)}_{\text{heading}} - \underbrace{0.005}_{\text{time}} - \underbrace{0.3(1 - l_{\min})^2 \mathbf{1}[l_{\min} < 1\text{m}]}_{\text{proximity}} + r_{\text{terminal}} \quad (1)$$

Terminal rewards: $r_{\text{collision}} = -10$, $r_{\text{success}} = +20$. All values are kept in $[-10, +20]$ to prevent critic divergence (see Section III).

II. A2C ARCHITECTURE

Both Actor and Critic share the topology $\mathbb{R}^{14} \rightarrow \text{ReLU}(256) \rightarrow \text{ReLU}(128)$, with separate output heads (Tanh(2) and $\mathbb{R}^1$ respectively). A learnable log $\sigma \in \mathbb{R}^2$ (init -0.5 , clamped to $[0.05, 1.0]$) parametrizes the Gaussian policy [3].

The composite loss per episode rollout is:

$$\mathcal{L} = -\frac{1}{T} \sum_t \log \pi_\theta(a_t | s_t) \hat{A}_t + \frac{0.5}{T} \sum_t (V_\phi(s_t) - G_t)^2 - \frac{0.01}{T} \sum_t \mathcal{H}[\pi_\theta(\cdot | s_t)] \quad (2)$$

where G_t uses $\gamma = 0.99$ with bootstrap, and $\hat{A}_t = (G_t - V_\phi)/(\text{std} + \epsilon)$ is normalized. Adam optimizer ($\text{lr} = 10^{-4}$) with gradient clipping ($\|\nabla\|_2 \leq 0.5$).

III. PHASE 1 RESULTS

A. Training Convergence

Fig. 1 shows the training curves over 3000 episodes with 3 dynamic obstacles and random goals. Table I summarizes key checkpoints.

TABLE I
A2C TRAINING METRICS (SELECTED EPISODES).

Ep.	$\bar{R}$	Success	L_a	L_c
100	4.1	2%	-0.007	7.6
500	5.6	0%	0.005	4.9
900	5.9	17%	0.024	20.8
1000	8.5	23%	-0.003	23.4
1500	12.5	35%	-0.004	25.2
2000	23.8	65%	0.007	24.8
2500	22.0	70%	-0.012	35.6
3000	23.2	75%	-0.018	29.7

Three learning phases are distinguishable: random exploration (ep. 0–800, success < 4

B. Reward Shaping

The original design used $r_{\text{col}} = -150$, $r_{\text{suc}} = +200$, and progress $\times 120$, yielding $L_c > 1500$ and policy collapse (robot only spinning). The corrected scale $[-10, +20]$ with quadratic proximity penalty instead of the reciprocal $0.5/(l + 0.01)$ allowed stable critic convergence and the actor to learn forward motion.

IV. DAGGER PIPELINE IN GAZEBO

The deployment uses ROS 2 [1] architecture via the node execution command `ros2 launch ros_gz_sim_demos diff_drive.launch.py` with the `vehicle_blue` robot subscribing to `/model/vehicle_blue/odometry` and publishing to `/model/vehicle_blue/cmd_vel` inside the Gazebo simulator [2].

Each DAgger iteration [4] follows: (1) A2C policy drives the robot autonomously; (2) the human operator intervenes (keyboard W/A/D) when collision is imminent, capturing

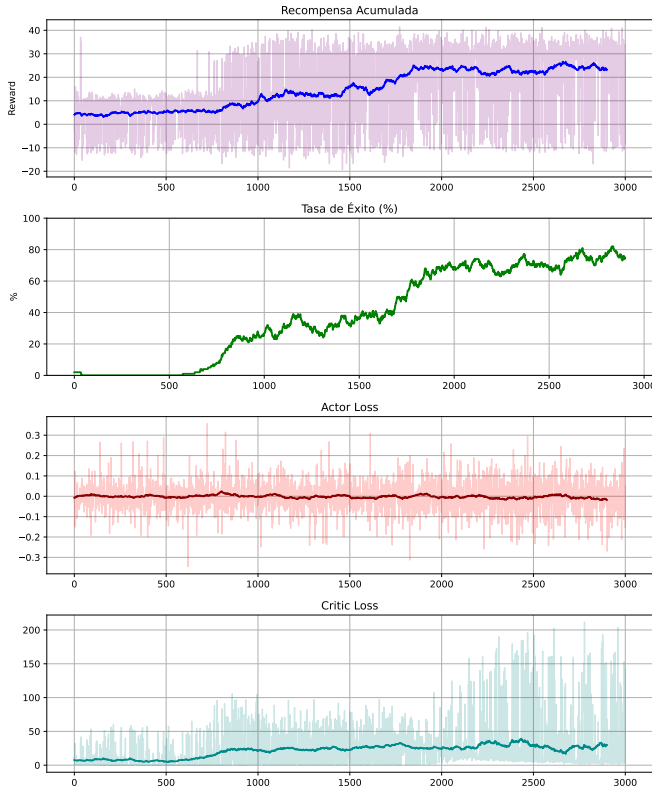


Fig. 1. Training curves: reward, success rate, L_{actor} , L_{critic} . Thick line = 5-episode moving average.

(s_t, a_t^*) pairs; (3) `finetune.py` minimizes $\mathcal{L}_{\text{imitation}} = \|\pi_\theta(s) - a^*\|^2$ on the buffer, updating the policy online.

Goal markers (green spheres) are spawned/removed via `gz service /world/diff_drive/{create,remove}`, the correct API for Gazebo Harmonic (gz-sim8). The legacy `gz entity -delete` from gz-sim7 fails silently in this version.

V. CONCLUSION

The A2C agent reaches a 75% success rate in simulation with stable convergence. The DAgger loop in Gazebo enables interactive policy correction without restarting training. Key design decisions—uniform reward scale, normalized state features, and quadratic obstacle penalty—were essential to avoid policy collapse. Phase 2 will analyze how ground friction, sensor noise, and motor latency widen the Reality Gap on the physical TurtleBot.

APPENDIX: DIGITAL RESOURCES

To provide reproducibility and full verification of the experimental validation, the engineering resources are explicitly shared below:

- **Source Code Repository:** The implementation scripts, configuration files, and training nodes are available at:
<https://github.com/ricardoamiel/RL-Autonomous-Robot>

- **Demonstration Video:** The complete recorded footage showing the baseline model flaws, online interactive corrections, and successful obstacle avoidance maneuvers can be accessed at:
<https://drive.google.com/drive/folders/1B9fmxxhI6fptCRY4967GAo6sUedK0XVJ8?usp=sharing>

REFERENCES

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022, url="https://doi.org/10.1126/scirobotics.abm6074".
- [2] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3, 2004, pp. 2149–2154, url="https://doi.org/10.1109/IROS.2004.1389727".
- [3] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International Conference on Machine Learning (ICML)*, 2016, pp. 1928–1937, url="https://proceedings.mlr.press/v48/mniha16.html", doi="https://doi.org/10.48550/arXiv.1602.01783".
- [4] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635, url="https://proceedings.mlr.press/v15/ross11a.html", doi="https://doi.org/10.48550/arXiv.1011.0686".